AI INFRA / SOURCE CODE STUDY

DeepInfer

仓库深度解析

从模型专用执行路径，
到置信度驱动的推理调度

重点阅读

Attention DP / CP ·YoCo bounded replay
Mega MoE ·EPLB ·DSpark / DFlash
MTPVarlen ·ZOS ·Batch invariance

核心判断

优化的对象不只是一次 GEMM，而是整条执行链：
哪些 token 值得验证、怎样摆放专家、
哪些 query 可以少算、哪些状态必须留在设备端。

分析对象：deepseek-ai/DeepInfer
固定提交：8b16149741a4935e1dec8cea27853f273b374c5b
核查日期：2026-10-20
依据公开源码与文档；未编译、未运行设备基准。

阅读导航

1. 先给结论：它是推理执行核心，不是完整的在线产品 1
2. 从入口到设备：先建立仓库的执行地图 1
3. 并行布局：Attention DP/CP 与 MoE EP 不是机械相乘 3
4. Attention 主线：稀疏索引、压缩 KV 与融合路径共同决定成本 4
5. YoCo bounded replay：减少后半段计算，不是丢掉整段历史 4
6. MoE 执行：Mega MoE 与 DeepEP 路径要分开看 6
7. EPLB：它优化的是历史最慢 rank，而不只是平均专家负载 7
8. DSpark / DFlash：草稿生成与验证预算是两个问题 8
9. 置信度调度：把接受概率与实测运行时间连接起来 8
10. GPU 预算分配 kernel：一个很具体的算子研究入口 10
11. Zero-overhead scheduling：让 CPU 调度尽量退出关键路径 11
12. Batch invariant、固定 seed 与逐位一致：三个不同层次 12
13. Kernel 与低精度：真正的接口包括 stride、scale 和缓存状态 12
14. 性能数据怎么读：这里最需要防止“数字口径迁移” 13
15. 正确性、模型质量与结构化输出：分别验收 14
16. 复现建议：在 Linux 设备机运行，Windows 负责连接与管理 15
17. 最适合你的三个研究切入点 16
18. 最后判断与源码阅读路线 17
附录：来源与阅读范围 18

1. 先给结论：它是推理执行核心，不是完整的在线产品

DeepInfer 最值得研究的，不是“又有一个推理框架”，而是 DeepSeek 如何把模型结构、专家放置、投机验证、缓存布局、CPU 调度和 GPU kernel 组成一条共同优化的执行链。

这次开源版本明确提供离线推理，当前 README 列出的模型是 **DeepSeek-V4.1-Flash**，硬件是 **NVIDIA SM100 GPU 与华为 Ascend 950 NPU**。它包含服务大规模推理所需的若干核心机制，但完整 KV 缓存服务、VE/Prefill/Decode 分离部署，以及 DP 副本之间的外部请求负载均衡，不在本次发行范围内。S01

因此，应把它定位为：**面向特定模型与硬件、具有生产背景的推理运行时源码**，而不是开箱即用、支持任意模型的通用在线服务替代品。这里的“生产背景”来自项目声明，不等于本文独立验证了其线上运行情况。

先分清的概念	本仓库能确认的范围	不能直接推出的结论
开源生产推理框架	公开离线引擎、调度与模型执行路径	完整线上平台已经公开
没有完整 KV cache system	本地 KV、block 管理与状态缓存仍然存在	模型执行不使用 KV cache
支持批次不变性	NVIDIA 路径有专门配置与回归测试	所有硬件、版本、并行布局逐位相同
支持投机解码	DSpark/DFlash 草稿与验证调度已实现	任意负载都更快，或吞吐等于实际产出
多个优化算子库	框架调用并编排这些实现	所有底层 kernel 都在本仓库重新实现

阅读边界。本文固定于 2026-10-20 核查到的提交8b16149741a4935e1dec8cea27853f273b374c5b，包版本为0.1.0。分析来自 README、运行示例、配置、关键 C++/Python/TileLang 源码及测试的静态阅读；没有编译项目、加载模型、运行 GPU/NPU 基准或复现作者质量评测。没有逐行审计全部仓库，也未将无法读取的二进制 Q&A 附件及性能曲线图片作为证据。所有公式推导、教学示例和实验建议均与源码事实区分。S00、S02

2. 从入口到设备：先建立仓库的执行地图

2.1 五层结构，而不是一个 Python forward

上层离线 API 接受请求，LLMEngine 管理序列与迭代执行；C++ 组件承担调度、元数据、block 管理等操作；worker 把执行计划变成设备输入；V4 模型运行时负责具体计算与缓存访问；最后由 DeepGEMM、FlashMLA、DeepEP、TileKernels 和本地 TileLang 算子完成重计算路径。S01、S03

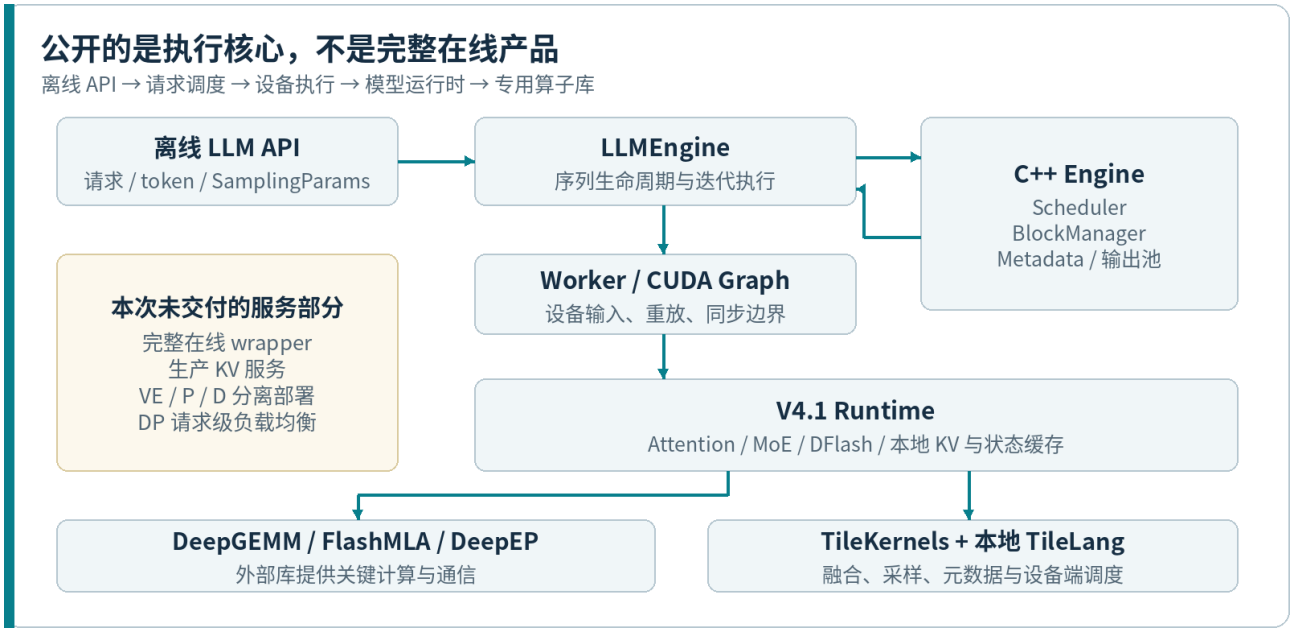


图 *1*：公开执行核心与完整服务系统的边界。依据 *README* 和引擎代码重绘，不是官方架构图。*S01*、*S03*

目录	读它是为了回答什么问题
deepinfer/entrypoints、engine	请求怎样进入，引擎如何管理一次迭代?
csrc/engine	调度、序列、验证长度和设备输入如何组织?
deepinfer/worker	设备初始化、执行、缓存与图重放如何衔接?
model_executor/models	模型结构怎样变成实际执行路径?
model_executor/eplb	逻辑专家怎样复制、放置和映射?
model_executor/layers	Attention、GEMM、采样和 TileLang 的接口是什么?
examples、tests、eval	演示、性能测量、数值回归和模型质量如何分别验证?

这张表是阅读导航，不表示每个目录下所有文件都已审计。后文重点跟踪的源文件均在文末列出固定提交链接。

2.2 为什么引擎实现细节也属于性能设计?

LLMEngine 不只是包装模型。它还持有序列元数据池、请求输出池、本地 block 管理器，以及需要延迟释放的资源。DeferredRelease 将部分回收推迟到下一轮pre_sampler_sync；涉及克隆缓存的 block/state 时，要等待对应事件完成，再解除占用。S03

这说明优化异步执行时，必须同时回答三个问题：**谁产生数据、谁最后读取它、什么时候才能回收对应存储?** 单独减少一个同步点，却提前释放仍在使用的 buffer，会把性能改动变成状态错误。

从源码还能看到项目对设备初始化时机的重视：包初始化避免直接导入 torch 或初始化设备，平台对象延迟创建。多进程运行时，先初始化 CUDA/NPU 再修改可见设备，可能使预期的设备分配失效。S02这不是模型数学问题，却直接影响运行时可靠性。

3. 并行布局：Attention DP/CP 与 MoE EP 不是机械相乘

3.1 正确理解 DP、CP、EP 的关系

ParallelConfig 在未显式给出 DP 时，默认取 `max(1, EP // CP)`；模型并行大小等于 CP，并要求 `CP × DP` 能被 EP 整除。若 Engram 并行不额外扩大运行规模，典型关系是：S04

$$W = CP \times DP, \qquad EP \mid (CP \times DP).$$

不是无条件使用 $W = DP \times CP \times EP$。EP 是同一批物理设备上的专家分组，而不是额外再乘一组卡。完整实现还考虑 Engram 并行大小，world_size 取相应规模的最大值，并做整除检查，因此上式有明确前提。

例如，下面是根据配置约束构造的布局示意，不是已经跑通的容量承诺：

物理规模	Attention 侧	Expert 侧	直观含义
8 卡	DP8、CP1	EP8	每卡处理自己的请求，专家阶段在 8 卡间交换 token
8 卡	DP4、CP2	EP8	每份请求上下文由 2 卡处理，专家仍可跨全部 8 卡
8 卡	DP8、CP1	EP2	同一设备集合中组织更小的 EP 子组

这与前一篇 Megatron 报告的一个核心思想相通：**Attention 与专家层的最佳分布方式不必相同**。但它不是训练侧 Parallel Folding 的同一套实现，也不能照搬训练时 TP/PP/EDP 的参数组合。

3.2 Balanced CP：前后配对，不是简单连续切片

prepare_cp_mirror_ref 给出了明确的 zigzag 定义：将每条序列划成约 $2C$ 个块，CP rank r 取得第 r 块和第 $2C-1-r$ 块，必要时补齐。CP4 的对应关系为：S05

$$(0,7), \quad (1,6), \quad (2,5), \quad (3,4).$$



图 *2*：*CP4* 的镜像分块。对于因果注意力，靠后的 *query* 通常能看到更多历史，前后配对有助于减轻偏斜；稀疏选择、压缩比例和 *padding* 仍会影响实际负载。*S05*

源码不仅生成mirror_cp_index，还生成逆映射及不同压缩比例下的 key 范围。这些元数据负责保持原始位置、重排位置与合法 KV 范围之间的关系。**布局变了，位置语义不能跟着丢掉**。

4. Attention 主线：稀疏索引、压缩 KV 与融合路径共同决定成本

4.1 不是“调用一次 FlashAttention”那么简单

V4AttentionBase 处理投影、归一化与 RoPE 等公共操作；V4Attention 叠加压缩 KV、稀疏索引、滑动窗口、YoCo 和跨层共享；DFlash 还拥有独立的 draft KV 路径。后端接口分别处理 sparse prefill 与带缓存的 sparse decode。S06、S07

因此，Attention 的性能至少包含三个层次：**选择哪些历史位置、怎样组织和存储这些位置、怎样在设备上计算选中的 attention**。优化最后一层的 kernel，并不意味着前两层的数据准备和访问已经没有成本。

一个便于理解的分解是：

$$T_{\text{attn}} = T_{\text{proj}} + T_{\text{index}} + T_{\text{KV access}} + T_{\text{attention}} + T_{\text{out}}.$$

这是本文的分析分解，不是仓库提供的时间测量。多个部分可能融合或重叠，不能直接把单独测量值相加当成端到端延迟。

4.2 融合不仅消除 launch，也改变数据布局

源码具有fused_norm_rope_attn_rope_cast 路径。在适用配置下，后端可以接收尚未完成对应 norm/RoPE 的、已按要求重排的 query，并返回 FP8 数据与 scale，供后续投影使用。权重和 scale 也会为该布局进行变换。S06、S07

这类融合的收益不止是“多个 kernel 合成一个”：它还可能避免写回中间 BF16 张量、减少重复读取，并让下一步直接消费量化后的结果。代价是**接口约束更强**。替换 kernel 时，只对齐 shape 和 dtype 不够，还要检查权重排列、scale layout、逻辑 head 数、位置编码，以及输出是否已经量化。

普通后端会将不在支持集合中的 head 数补到 64 或 128，再切回逻辑 head 范围；超过支持范围则报错。S07这意味着“参数看起来可配置”并不等于任意 head 形状都具有相同效率。

4.3 causal=False 不能脱离索引管线阅读

稀疏 decode 后端调用中出现causal=False，同时传入显式 KV 索引和有效长度。S07不能看到这个标志就认定语言模型在读取未来 token：稀疏场景的可见集合可以由上游索引约束，而不是让底层 kernel 再生成普通三角 mask。

正确的检查对象应当是：**索引是否只包含允许访问的位置、序列之间是否隔离、压缩位置与原位置是否一致、回退和缓存复用时是否保持这些条件**。本文没有对整个索引生成链做形式化正确性证明。

5. YoCo bounded replay：减少后半段计算，不是丢掉整段历史

5.1 先理解 producer 与 consumer 的分工

源码用 `n_prefill_layer < num_hidden_layers` 判断 YoCo 是否启用，并把之后的层视为 consumer。根据跨层共享拓扑，consumer 可以消费前面提供的共享 hidden 或 KV，而不是每层都独立重做相同的历史构建。S08

在这个结构基础上，bounded replay 可以限制 consumer prefill 需要重新处理的尾部 query。**原始历史仍通过 producer 计算和缓存供后续使用；不是把用户输入直接裁成最后 140 个 token**。S01、S08

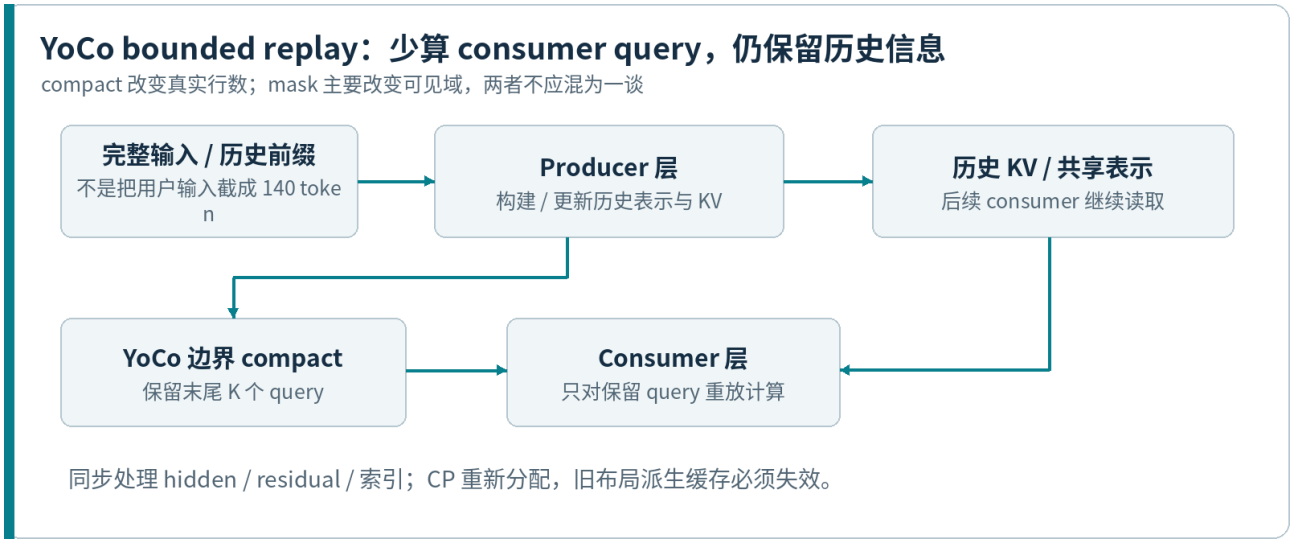


图 *3*：*bounded replay* 减少 *consumer* 的 *query* 工作量，历史可见性由模型结构与缓存共同维持。*S08*

5.2 mask 与 compact 的区别是真实工作量

mask 保留 query 行，只收窄可见域；compact 真正移除截断区的 query 行。因此，只有限制 mask 并不自动减少所有线性层、MoE 等对这些 query 的计算。源码为 compact 做了边界 gather，还会同步处理 residual、mHC 的中间状态以及跨层索引。S08、S09

CP 开启时，保留的尾部 token 可能集中在少数 rank 上。代码会先提取各 rank 的贡献，再 all-gather，最后按 compact 后的 CP 布局重分配。布局改变后，派生的 candidate-MQA 调度缓存被清空，避免继续使用旧行坐标。S08

这是很值得借鉴的工程细节：**缩小张量不能只改 shape；所有依赖旧布局的元数据和缓存，都必须随之更新**。

5.3 为什么保留长度还要考虑回退?

配置要求：

$$K_{\text{keep}} \geq W_{\text{SWA}} + R_{\text{backtrack}}.$$

其中，运行时回退预算至少覆盖 `nextn + 1`；README 给出默认 V4.1 场景的keep_tokens=140，验证示例使用窗口 128。S01、S09、S10这些不是对所有配置都成立的万能常数。改变最大草稿长度或回退预算后，必须重新检查保留长度。

仓库提供两层验证：L1 检查元数据，包括可见 key、store slot、压缩范围；L2 使用兼容 checkpoint、固定 teacher-forcing 轨迹，逐元素比较 logprob 和 top-k，还可检查 DFlash confidence。S09

注意证据边界：mask/compact 等价测试不等于证明 “任意模型删除历史 query 都正确”。它验证的是受支持结构、同一可见策略下两种实现的等价性。

6. MoE 执行：Mega MoE 与 DeepEP 路径要分开看

6.1 逻辑专家、物理副本和 shared expert

源码区分三种数量：router 选择范围内的逻辑专家 E，为负载均衡增加的副本 R，以及设备上实际存放的物理专家 $E + R$。每卡的物理专家数量为：S11

$$E_{\text{local}} = \frac{E + R}{EP}.$$

物理数量必须能被 EP 整除。示例中的额外专家数量 32 是下限，实际值会向上取整；主模型与 DFlash 的专家数量分别处理。S12Shared expert 则是独立计算分支，不能直接把它等同于 EPLB 冗余副本。

当前模型容器明确约束 router 使用sqrtsoftplus、noaux_tc 和归一化的 top-k 权重。gate 权重保持 BF16，运行时投影输出可以为 FP32；文本和图像 token 还有分开的校正 bias。S11、S13因此，不能把它当成对所有 softmax/sigmoid MoE 都无条件通用的路由器。

6.2 同一 NVLink 域内的核心分支：Mega MoE

DeepseekMoE 根据平台、精度组合及 EP 组是否位于同一个 NVLink 域，决定使用 Mega MoE。支持的组合包括fp8xfp4、fp8xfp8 与bf16xbf16，但具体分支还有平台条件。S13

Mega 路径使用 symmetric buffer：gate 的输出写入其中，输入的 norm/cast 也可以直接把量化数据和 scale 写到目标 buffer，随后调用 DeepGEMM 的 mega-MoE 接口；存在 shared expert 时还可以传入对应权重。S13

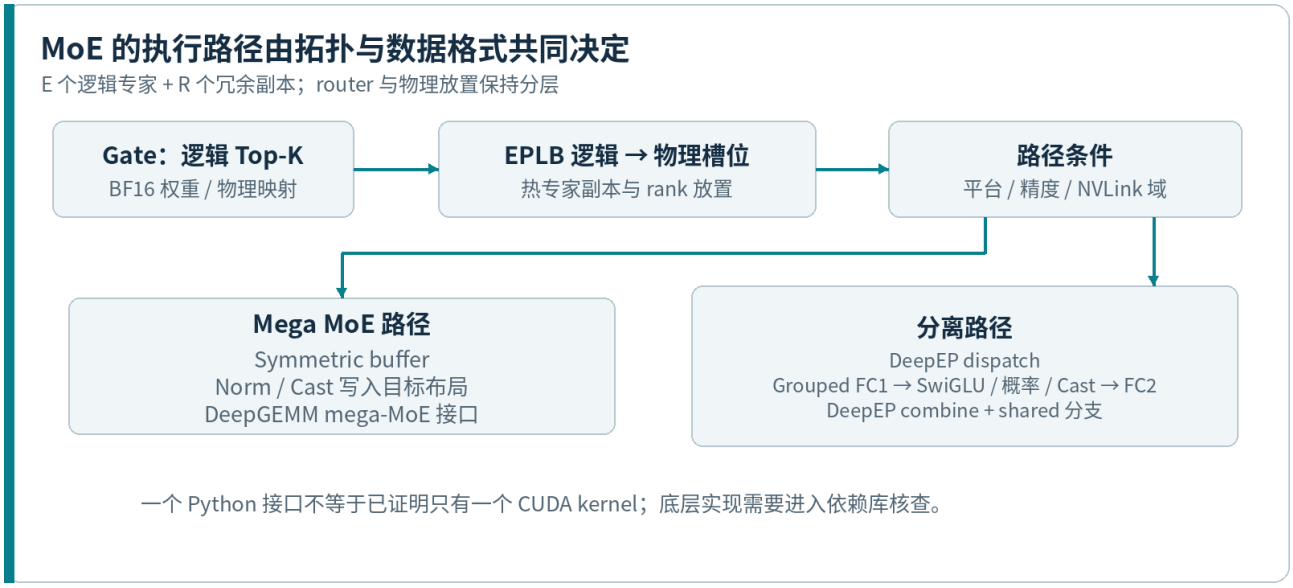


图 *4*：*MoE* 的两条公开调用路径。*Mega* 接口的底层 *CUDA* 实现在依赖库中；本图不把一个 *Python* 调用误写成已经审计过的 *“单个 GPU kernel”*。*S13*

这个分支对理解官方性能尤其重要：**不能默认 DeepInfer 所有 MoE 层都执行同一条 “DeepEP + 两次 GEMM” 链路**。拓扑变化可能连执行路径都改变，而不只是让同一个 All-to-All 变慢。

6.3 分离路径：概率乘法已经前移到 FC2 之前

另一条路径是 dispatch、Grouped FC1、SwiGLU 与 cast、Grouped FC2、combine。SwiGLU/cast 接口接收topk_weights，说明路由权重在第二个线性层之前应用。S13

在无 bias 的线性层条件下，实数代数满足：

$$p\,W_2\phi(W_1x) = W_2(p\,\phi(W_1x)).$$

这与此前 Megatron 报告中的概率前移思想一致，但在这里服务于推理融合、量化及组合路径。**代数等价不代表 FP8/BF16 舍入顺序或逐位结果相同**。更不能把概率继续移过非线性函数而不改变含义。

分离路径的 dispatch 还有值得关注的开关：部分 prefill 情况使用 CPU 同步，而 compact 模式避免该同步，代价可能是额外 GPU 内存。S13这再次说明，“异步调度”不等于整个框架在所有路径都没有 host synchronization。

7. EPLB：它优化的是历史最慢 rank，而不只是平均专家负载

7.1 为什么“总访问量一样”仍然会慢?

假设专家 A、B 总 token 数相近，但它们经常在同一轮变热。把两者放在同一张卡，即使长期平均数平衡，每一轮仍可能出现明显长尾。相比之下，交替变热的专家更适合共置。

policy.py 保留形状为“历史 step × layer × logical expert”的统计，并分两步执行：先分配冗余副本，再安排物理专家的 rank 放置。S14

7.2 将代码中的目标写成公式

固定某一层，设 $n_{t,e}$ 是专家 e 在历史 step t 的 token 数，c_e 是副本数，$\pi(p)$ 是物理副本 p 对应的逻辑专家。放置目标可以近似写成：

$$\min_{\text{placement}} \sum_t \max_r \sum_{p\in r} \frac{n_{t,\pi(p)}}{c_{\pi(p)}}.$$

每个 rank 还受固定的物理槽位容量约束。该公式是对_batch_greedy_indices 注释与实现的整理：每次放置一个物理专家时，尝试各 rank，选择使历史逐步峰值总和较小的位置。S14

这与单纯最小化“所有 step 加起来后，各卡总 token 数的最大值”不同：**前者保留了时间上的共同热点信息**。副本分配阶段也使用历史负载来降低峰值，而不是无条件给每个专家增加一样多的副本。

7.3 实现取舍与边界

这里是贪心启发式，不是全局最优解。它依赖“过去的负载对未来有一定解释力”，而且还要付出专家搬运、额外显存、统计收集和映射更新的成本。本文没有测量这些成本，也未逐行审查shuffle.py 的传输实现。

对于推理，专家副本使用相同模型权重，不涉及训练中的独立 optimizer state 或副本梯度回收。其目标与训练侧热专家复制有相似之处，但状态管理负担不同。

另外，**内部 EPLB 已实现与完整服务的 DP 请求负载均衡未提供**不矛盾：一个决定 token 的专家执行位置，另一个决定请求进入哪个服务副本。S01、S14

8. DSpark / DFlash：草稿生成与验证预算是两个问题

8.1 名称与实际代码对应

README 使用 DSpark 描述相关加速能力；当前模型代码以DFlash 类组织草稿路径，并明确只支持对应的 speculative head。它从主模型选定层收集特征，投影后更新 main-token cache，再运行草稿层生成候选表示。S01、S15

草稿 forward 按固定步幅组织；采样仍逐草稿位置循环，能够根据前一个采样 token 施加可选的低秩 Markov 修正。模型同时输出每个位置的 confidence，用于估计草稿被接受的可能性。S15

因此，应区分：**草稿 hidden 如何计算、草稿 token 如何采样、主模型验证多少个草稿**。不能因为一个阶段使用块状计算，就说整个流程完全没有顺序依赖。

8.2 草稿与主模型缓存是不同生命周期

DFlash 的主缓存投影为特定连续布局保留额外权重视图；其 draft KV 与主模型执行有专门衔接。源码注释解释，在主 forward 阶段，某些最终会被拒绝的 draft 位置仍可能先写入，随后在任何相关读取前被正确覆盖。S15

这是一种状态生命周期设计，不是 “拒绝草稿后可以忽略缓存”。正确性依赖写入位置、接受数量、回退位置和下一次读取的先后关系。

此外，DFlash 拥有单独的 CUDA Graph runner，其重放范围包含广播、固定步幅元数据、草稿层、采样和 proposal pool 写入。S15主模型 graph 与草稿 graph 协作，并不代表请求数量或实际验证长度完全不变。

8.3 为什么固定草稿长度不是总能获得最好吞吐?

多验证一个草稿，会增加主模型工作量；如果它几乎不会被接受，收益可能低于成本。反过来，某些 graph bucket 已经有空余位置，增加少量高置信度草稿，可能几乎不改变执行时间。

所以正确的优化目标不是 “草稿越长越好”，而是：

$$\frac{\text{预计有效产出 token}}{\text{预计本轮执行时间}}.$$

MTPVarlen 正是在做这个取舍。S16

9. 置信度调度：把接受概率与实测运行时间连接起来

9.1 从单个位置的 confidence 到前缀收益

GPU 侧调度先对每条序列的 confidence 做累计乘积。用 $c_{i,j}$ 表示序列 i 第 j 个草稿的条件接受概率估计，得到：

$$a_{i,j} = \prod_{k=1}^{j} c_{i,k}.$$

$a_{i,j}$ 可以解释为 “前 j 个草稿均被接受” 的估计概率。若最多验证 d_i 个草稿，常用的期望接受前缀长度表达式为：

$$\mathbb{E}[A_i] \approx \sum_{j=1}^{d_i} a_{i,j}.$$

这是与代码累计乘积和期望产出计算相匹配的解释。实际 confidence 是否校准，仍然需要验证；它不是由类型或取值范围自动保证的真实概率。S16、S17

9.2 验证位置数与 graph 大小不一定相同

设本轮有 N 个请求，每个请求至少安排一个主 token，额外验证 $Q - N$ 个草稿位置。代码按收益排序候选，估计：

$$\widehat{E}(Q) = N + \sum_{(i,j)\in\mathcal{A}(Q)} a_{i,j}.$$

$\mathcal{A}(Q)$ 为被分配预算的草稿位置集合。实际执行选择能容纳 Q 的 graph bucket，记为 $g(Q) \geq Q$。时间估计为：

$$\widehat{T}(Q) = T_0(g(Q)) + \max(0, \hat{s})\, g(Q) + \widehat{T}_{\text{sampler}}(N),$$

进而选择：

$$Q^* = \arg\max_{Q} \frac{\widehat{E}(Q)}{\widehat{T}(Q)}.$$

其中 T_0 来自启动时的 decode 性能曲线，$\hat{s}$ 是运行时对 forward 残差的修正，sampler 时间另行估计。上述记号是对源码的数学整理，不是新提出的算法。S16



图 5：验证预算是收益预测与执行成本预测的联合决策，不是固定的 “接受率阈值”。*S16、S17*

9.3 一个具体例子：更多验证可能更慢

以下数值全部为**教学假设**，不是 DeepInfer 基准。两条序列各有两个草稿，累计前缀概率分别为 $(0.9, 0.5)$ 和 $(0.7, 0.2)$。graph 大小 2、4、6 的 forward 时间假设为 1.0、1.2、1.8 ms，sampler 恒为 0.2 ms，不额外加斜率修正。

总验证位置 Q	对应 graph	预计产出	总时间 / ms	预计 token/ms
2	2	2.0	1.2	1.67
3	4	2.9	1.4	2.07
4	**4**	**3.6**	**1.4**	**2.57**
5	6	4.1	2.0	2.05
6	6	4.3	2.0	2.15

预算从 4 增加到 5 时，新增收益很小，却跨入更昂贵的 graph bucket。**最佳策略是在这里停下来，而不是验证所有草稿**。

9.4 运行时估计器不是固定离线 cost model

Sampler 使用小窗口中位数，优先选相同请求数的历史估计，不足时使用全局回退。Forward 修正使用最近样本，按 graph size 汇总中位数，再取跨 graph 的中位数；源码有样本门槛，并限制负斜率不把预测时间压到启动基线以下。S16

这与 TileSight 的分析式建模不同：**这里直接使用观测到的执行时间曲线与在线残差**。优点是贴近实际运行环境；限制是噪声、负载变化和模型误差都可能影响预算选择。

9.5 为什么源码特意使用两轮历史?

MTPVarlen 明确区分选择总预算与分配各请求预算的历史：总预算使用较旧的 N−2 轮信息，分配使用更近的信息；ZOS 下尚未完成 CPU 后处理的请求，则通过计数器与 GPU 修正保持关系。注释还强调，后续位置的 confidence 不应影响前面草稿是否被验证。S16

这不是一个可以随意“改成最新数据更聪明”的小优化。调度会改变验证决策的依赖关系；涉及投机采样时，应该检查这种依赖是否影响输出分布。**本文没有从采样器全过程证明分布保持性，也不会把这段注释当成完整证明**。

10. GPU 预算分配 kernel：一个很具体的算子研究入口

cuda/schedule_mtp_varlen.py 分两阶段执行。第一阶段读取上一轮 confidence，通过 request remap 找到对应序列，计算累计概率；新请求没有历史则按零收益处理。第二阶段，为每个候选位置计算其在所有候选中的排名，收益相等时按较小的扁平索引优先，最后把各请求的 query 长度设为“1 + 获得的额外位置数”。S17

由于累计前缀概率单调不增，且相同序列的早期位置具有更小索引，这种排序也有助于维持前缀选择。该解释假设 confidence 在合法的概率范围内。

根据循环结构，若请求数为 N、最大草稿长度为 d，直接排名比较的工作量量级为：

$$O\big((Nd)^2\big).$$

这是本文对实现的复杂度分析，不是性能故障结论。对于较小的 d 和有限 batch，全比较容易并行，省去 CPU 往返和复杂排序阶段，可能反而合适。

可研究的替代方案包括稳定 GPU select、分块 top-k、前缀受限选择或分层阈值法。但必须同时保持：总预算不超限、每条序列前缀有效、相同输入的 tie-break 一致，以及与 ZOS remap 的关系正确。

对你的算子优化方向，最有价值的实验不是只展示这个 kernel 快了多少，而是回答：**它是否出现在真实 decode 的关键路径上？优化后能否减少整轮时间，还是增加了额外 launch、scratch buffer 或同步？**

11. Zero-overhead scheduling：让 CPU 调度尽量退出关键路径

11.1 “零开销”不是没有调度工作

朴素串行流程是 GPU 生成、把结果回传 CPU、CPU 更新序列并组下个 batch、再提交 GPU。理想情况下，单轮时间近似为：

$$T_{\text{serial}} \approx T_{\text{GPU}} + T_{\text{CPU}} + T_{\text{transfer}}.$$

异步调度希望把下一轮可提前完成的 CPU 工作与当前设备执行重叠，使暴露在关键路径上的时间接近二者的最大值，再加不能隐藏的边界成本。这只是解释性的理想模型，并非实测加速比。

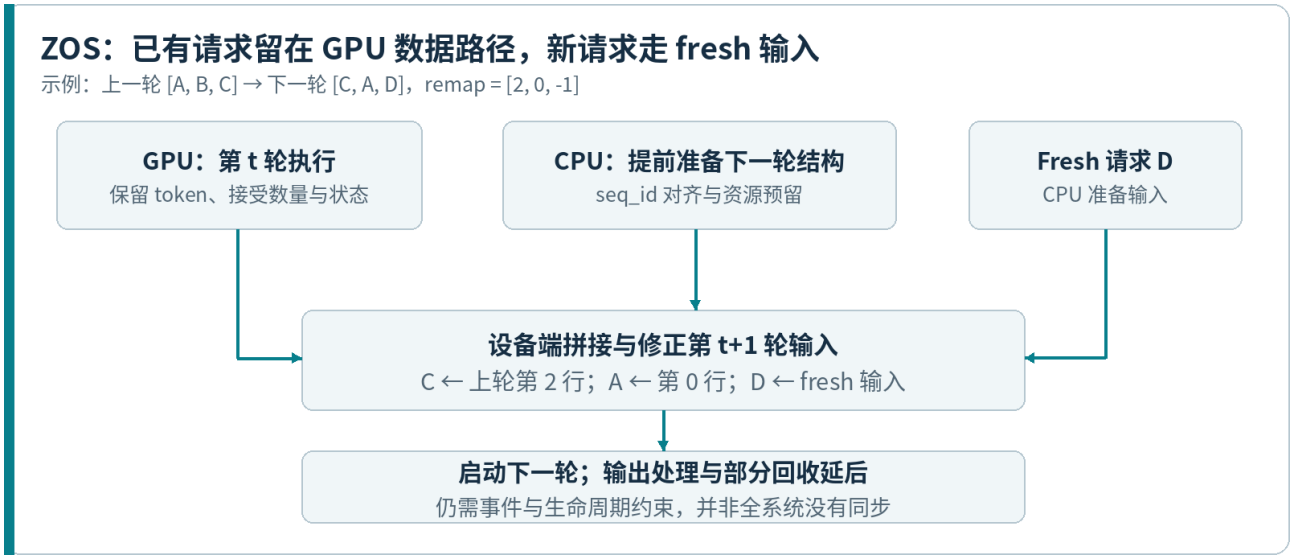


图 *6*：已有请求复用前一轮 *GPU* 输出，新请求使用主机准备的数据，*remap* 连接两轮 *batch*。*S18*

11.2 remap 是关键，不只是双缓冲

prepare_zos_inputs 比较前后两轮seq_id。顺序完全相同可以走快速路径；发生插入、退出或重排时，使用映射找到上一轮的位置，新请求标记为 −1。存在重叠时，CPU 只填充 fresh rows，继续请求的真实 token 来自前一轮 GPU 输出。S18

例如，上一轮顺序为[A,B,C]，下一轮为[C,A,D]，映射应为[2,0,-1]。如果只按行号复用，就会把别的请求的输出当成输入。

同样，提前调度时未知的实际接受数量，需要在设备上修正 query/key 长度，并为最大可能的在途 token 预留缓存容量。系统并非消除了依赖，而是把依赖放到可以重叠和就地更新的位置。

11.3 默认值必须读到初始化入口

环境变量定义中的默认值不一定是用户入口的最终默认。源码搜索显示EngineArgs 会按 engine type 设置DEEPINFER_ZERO_OVERHEAD_SCHEDULING，并重新装载对应描述符；worker 也区分 prefill、非 ZOS decode 与延后一轮处理的 ZOS decode。S19

所以不要仅凭environ.py 中一个布尔描述符就判定 README 的 “默认开启” 错误。复现实验应显式记录最终环境和 engine type，最好每个对照启动独立进程。

12. Batch invariant、固定 seed 与逐位一致：三个不同层次

12.1 为什么相同请求换个 batch 可能改变结果?

浮点计算的归约顺序、kernel 配置、分块以及通信组合方式可能随 batch 或物理布局变化。即使数学表达式相同，结果也未必逐位相同；如果差异恰好发生在 router top-k 或采样边界附近，还可能改变后续离散路径。

DeepInfer 的 BI 模式并不只设置随机种子。Worker 会让 DeepGEMM 与 TileKernels 使用确定性算法，Attention 后端也有专门的 batch-invariant 参数或分支。README 明确把完整 BI 功能限制在 NVIDIA 路径，NPU 初始化还会拒绝开启该选项。S20、S07、S01

层次	想保证什么	仅固定 seed 是否足够?
Forward batch invariance	同一前缀在不同 batch 下有相同数值结果	不够，还受 kernel 与归约影响
随机数归属一致	同一序列、同一位置使用对应随机流	需要逐序列 seed 与位置管理
自由生成一致	相同输入和采样参数产生相同输出轨迹	还要求前两层及验证决策一致

README 给出的确定性采样组合是 **BI=1、MTP_VARLEN=0、固定seq_seed**。S01关闭变长验证，不等于必须关闭全部投机解码；为了定位问题，实验上可以先从nextn=0 的最小路径开始，再逐项恢复功能。

12.2 一个很容易看错的测试：单条重跑使用了 golden tokens

test_batch_invariant.py 先批量生成，再针对每个请求单独运行，并把第一次输出传给golden_token_ids。随后精确比较 token IDs、token logprobs、top token IDs 和 top logprobs。S21

这个测试的价值在于：**固定生成轨迹后，隔离批次变化对 forward 数值结果的影响**。它不是一次“两个完全独立随机生成过程自然相同”的测试。

这不是测试缺陷，而是测试目标不同。要验证自由生成确定性，应额外设置相同逐序列 seed，在无 golden 强制的情况下，对照 batch 大小、顺序、到达时间、取消请求、EPLB 布局及 graph bucket。

12.3 对 Router Replay / RL rollout 的意义

BI 与路由重放都与稳定训练—推理闭环有关，但不要混淆：前者试图控制不同批次下的数值执行差异，后者固定或复用逻辑路由。**路由相同不自动保证所有 logprob 一致，forward 一致也不自动保证随机采样轨迹一致**。

对于你的工作，值得借鉴的是分层测试：先固定前缀测 logprob，再检查逻辑路由，最后检查自由采样轨迹和接受长度。这样更容易定位误差来自模型算子、MoE 映射、随机数，还是调度状态。

13. Kernel 与低精度：真正的接口包括 stride、scale 和缓存状态

13.1 不要只按文件名判断“新算法”

ReplicatedLinear 最终调用 DeepGEMM 的 BF16 或 FP8/FP4 GEMM。框架层的重要工作是安排参数、输出布局、cast、scale 与累加目标，而不一定重新实现底层 MMA。S22

当前 FP8 cast 使用 32 元素粒度、E4M3、packed UE8M0 scale 与相应布局。输出行步幅按 32 字节对齐：实际存储宽度可以大于逻辑输出宽度，再返回逻辑视图。S22

这对小 N / 小 M kernel 替换很重要。一个输出 shape 正确的张量，仍可能因 stride 不满足契约而触发额外复制或直接失败。实现对接至少要核对 **逻辑 shape、物理 stride、scale 排列、累加精度、输出别名与生命周期**。

13.2 FP4 权重不是任意模型“一键无损压缩”

MoE 权重处理对 FP4 路径检查 QAT checkpoint 的特定条件，再转换或加载运行时 packed 布局。S13所以，看到 FP8×FP4 路径，只能确认该格式的支持方式，不能推出任意 BF16/FP8 checkpoint 都可以无精度损失地改成 FP4。

另外，Mega MoE、分离 Grouped GEMM、Attention fused 路径的 scale 与缓冲契约并不相同。应当把量化视作一条端到端数据通路，而不只是把某个张量的 dtype 改小。

13.3 统一接口不代表 NVIDIA / Ascend 完全同构

仓库为 NVIDIA 和 Ascend 提供平台分支以及对应依赖；部分 TileLang wrapper 共享签名，实际 kernel 走不同实现。完整 BI 当前不支持 NPU，就是一个明确的功能差异。S20、S05、S23

因此，跨平台比较应分别记录后端、版本、量化格式和启用功能，不能仅凭同一 Python 函数名就假定相同算法或相同数值保证。

14. 性能数据怎么读：这里最需要防止“数字口径迁移”

14.1 官方规模描述不等于最小部署要求

README 的性能图分别使用 **4 张 SM100 做 prefill、64 张做 decode，且各自在单一 scale-up 域内**。S01这说明了展示结果的拓扑前提，不证明其他卡数不能运行，也不保证普通跨节点网络能重现相同路径与性能。

本文未读取这些图片上的具体坐标，不提供从图中猜出的吞吐点，也不据此宣称对 vLLM/SGLang 有某个未经控制变量验证的倍数优势。

14.2 benchmark 的 decode 吞吐首先是满接受口径

示例明确给出：S12

$$\mathrm{TPS}_{\mathrm{reported}} = \mathrm{SPS} \times K \times (1+d)/CP,$$

其中 K 是**每个 DP rank** 的请求数，d 为 nextn，SPS 为 steps/s。这里使用每步每请求 $1+d$ 个 token 的满接受计数，**不等于统计出来的实际接受 token 数**。

为什么除以 CP 而不是再除 EP? 在 benchmark 的常见布局下，$DP = EP/CP$，整组产出按 DP 倍数增长，而设备数为 EP，因此化简为 K/CP。不能把一个 “per-DP-rank batch” 错当成全局 batch。

真实吞吐更稳妥的定义是:

$$\mathrm{TPS}_{\mathrm{real}} = \frac{\sum \text{实际提交给用户的有效输出 token}}{\text{纳入开销的墙钟时间} \times \text{设备数}}.$$

拒绝的草稿、仅供回退的状态或 padding 都不应被当作有效输出。

14.3 Pareto 图可以使用外部接受率假设

绘图代码采用：

$$\tau = 1 + d\,a,$$

并把每用户速度与每卡吞吐都乘以 τ。a 来自--draft_acceptance，默认 1.0；脚本示例中的 0.75 是用户给定的参数，不是绘图程序自动测出的真实接受率。S24

要把 a 用作有效估计，需要其含义与“平均每个生成草稿位置贡献多少已接受 token”一致；不能把某个条件位置的接受概率随意塞进去。前文控制器使用的累计前缀概率与这里的聚合标量也不是同一个对象。

代码拒绝直接按这一口径绘制 VARLEN=1 数据，提醒不能对已经受变长行为影响的读数再次机械乘相同系数。S24

14.4 稳态洁净窗口与线上服务指标不同

示例将 warmup、日志段、测量段和 profiler 段分开，并丢弃撞上 MoE LB 统计 flush 的时间间隔。S12这有助于观察稳定执行性能，但会排除部分真实系统成本。

因此，至少应同时报告两类结果：**洁净稳态吞吐**与**包含统计、重排、输入处理和实际接受率的墙钟吞吐**。前者分析 kernel/调度效率，后者更接近系统容量。在线场景还需要排队、TTFT、请求完成时间与 p95/p99 等指标，不能由 step 均值直接替代。

Prefill 横轴是测量窗口的平均 step 延迟，不是完整请求的 TTFT。其负载会复用缓存前缀，仅计算非缓存后缀；比较其他实现时必须匹配缓存命中条件。Vision encoder sweep 只计设备侧 pad、ViT、下采样和 aligner，排除了 CPU 预处理与 H2D。S12、S24

14.5 曲线处理也会影响结论

plot_pareto.py 对重复的扫描参数保留最后一条结果，不是多次实验取均值或中位数；它会检查模型、EP、CP、上下文、nextn、冗余专家等固定配置的一致性。S24

这是一份很有用的实验工具，但正式比较仍应保存重复试验、离散程度及失败点。**同一条漂亮的曲线，不等于已经给出置信区间**。

15. 正确性、模型质量与结构化输出：分别验收

15.1 不同测试回答不同问题

测试或机制	可以检查什么	不应被解释成什么
PPL golden regression	指定平台、负载和输入的数值变化	更新基线后就自动证明新实现正确
BI teacher-forcing 对照	固定轨迹下跨 batch 的数值一致	所有随机生成配置都逐位相同
YoCo L1/L2	元数据及 mask/compact 的对应关系	任意结构都能安全截断历史
Regex / JSON / EBNF	输出满足指定形式约束	答案事实正确、推理正确
AIME / GPQA / HLE	特定模型与生成评测协议下的任务表现	框架本身使模型能力无条件提升

上述测试边界分别来自示例、BI 测试和评测说明。S12、S21、S09、S25

15.2 多次采样后平均正确率，仍可在估计 Pass@1

评测说明把 AIME 的每题 32 次采样、GPQA 的每题 8 次采样，用于计算平均单次正确率。它不是“32 次里只要对一次就算对”的 Pass@32。HLE 使用单次采样；生成和评分分开保存，缺失 rank 或样本时coverage_complete 不为真，headline 指标就不应视为完整。S25

作者给出 AIME 99.9%、GPQA 93.4%、HLE-Full 42.6% 的参考值，并说明存在随机波动。S25本文未复现这些数字；它们是指定 checkpoint 与协议的部署 sanity check，不是与其他框架严格配对后的质量优势。

如果修改采样器、概率计算、量化或验证调度，应该先做机制级检查，再看任务质量。最终分数近似相同，可能掩盖细粒度 logprob 或分布变化；反过来，小范围数值差异也不必然等于有意义的质量下降。

16. 复现建议：在 Linux 设备机运行，Windows 负责连接与管理

16.1 先确定可复现的对象

当前 README 的平台目标是 SM100 与 Ascend 950，不是“任何装了 CUDA 的电脑”。Windows 原生不是官方构建路径；更稳妥的做法是在 Windows 用 SSH/VS Code 连接 Linux 设备服务器。WSL 可以用于阅读和准备代码，但不能因此默认获得受支持的多卡拓扑与依赖环境。S01

B200 属于 SM100 家族，H200 属于另一代计算能力；即使下层算子库支持某张卡，也不等于这一固定提交的完整框架已支持它。对硬件的最终判断仍应以仓库和设备环境为准。S26、S01

16.2 固定提交，使用独立环境

以下命令在 **Linux 服务器 shell** 执行，不是 Windows PowerShell 语法：

```
git clone https://github.com/deepseek-ai/DeepInfer.git
cd DeepInfer
git checkout 8b16149741a4935e1dec8cea27853f273b374c5b
git submodule update --init --recursive

python3.12 -m venv .venv
source .venv/bin/activate
export DEEPINFER_ACCELERATOR=nvidia
./build.sh
```

除 Python 之外，还要准备满足要求的 GCC、CUDA、驱动与实际设备。源码中的 NVIDIA requirements 指定torch==2.13.0+cu132、flash-attn-4==4.0.0b34；构建脚本会安装依赖，并强制替换为nvidia-nccl-cu13==2.32.3。S27、S28**不要直接在现有训练或推理环境里试跑构建**。

固定主仓库提交后仍应记录 `git submodule status --recursive` 和最终依赖清单，因为运行结果依赖整个软件栈，而不只是 DeepInfer 的短 SHA。

16.3 先跑最小路径，再逐步恢复优化

```
export DEEPINFER_HF_MODEL_DIR=/path/to/DeepSeek-V4.1-Flash-1017
export DEEPINFER_BATCH_INVARIANT=1
export DEEPINFER_ENABLE_MTP_VARLEN=0
export DEEPINFER_USE_NEXTN_DECODE=0
python examples/offline_inference.py --ep 8
```

`--ep 8` 是仓库已有示例，不是对任意 8 卡机器都能加载模型的保证。显存、主机内存、模型文件和实际可见设备必须先满足要求。更小规模是否可行，应结合 checkpoint 与配置验证，而不是仅按活跃参数量估算。

接着检查目标机制：

```
python -m pytest tests/models/test_batch_invariant.py -s
python examples/yoco/yoco_equiv.py mask-compact
# L2 需要兼容 checkpoint，且有相应多卡资源：
bash examples/yoco/run_yoco_equiv_l2.sh \
  --model_name "$DEEPINFER_HF_MODEL_DIR" --ep 8 --cp 1
```

测试可能自行设置 nextn、BI 等选项，不能把调用前 shell 环境当成全部最终配置。S21、S09环境描述符存在惰性缓存；做消融时使用新进程，并保存实际解析后的配置。S10

16.4 性能基线先关闭投机，再测固定草稿与变长验证

单节点 8 卡示意：

```
export WORLD_SIZE=1 RANK=0
export CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
export DEEPINFER_ENABLE_MTP_VARLEN=0
export DEEPINFER_USE_NEXTN_DECODE=0
PROFILE=0 python examples/benchmark.py \
  --decode_num_requests 8 --context_length 32768 \
  --result_jsonl baseline.jsonl
```

这里WORLD_SIZE 在该示例 launcher 中表示节点数，不是习惯性填写的 GPU 进程数。S12跨节点必须按项目 launcher 约定设置 rendezvous 与设备环境，不能直接把 torchrun 的同名变量语义套进去。

固定 nextn 的 sweep 示例：

```
export DEEPINFER_ENABLE_MTP_VARLEN=0
bash examples/sweep/run_sweep.sh decode \
  ep8-fixed-32k 32768 5 0.75
```

最后的 0.75 **只是绘图采用的接受率假设**。真实效果应另行累计有效 token。变长验证实验不要继续用这一固定草稿系数强行换算。S24、S29

建议每次记录：主仓库与子模块 SHA、checkpoint revision、平台/驱动、完整环境变量、EP/DP/CP、冗余专家实际值、BI、YoCo 模式、nextn、VARLEN、缓存状态、输入分布，以及冷启动和稳态的分开计时。

17. 最适合你的三个研究切入点

17.1 小 M 确定性 kernel：把“快”与“结果不随 batch 变”一起测

从linear.py、MoE gate、Attention backend 和 BI 测试开始，固定前缀与输入数值，对照不同 batch、图 bucket 和专家布局。先比较 logits/logprob/逻辑专家索引，再看自由采样输出。替换 kernel 时保持行步幅、scale layout 与累加语义。S22、S07、S21

可以研究确定性模式的额外开销究竟集中在哪里，但不能先假定是 GEMM。验收至少包括逐位或误差阈值、单算子延迟、整轮延迟，以及不同形状下的回归覆盖。

17.2 GPU 验证预算分配：用同样结果减少调度成本

围绕第 10 节的全比较排名实现，设计等价的稳定选择算法。测试请求数、草稿长度、同分比例、fresh/remap 混合比例及预算边界。先保证输出seqlens_q 与参考实现一致，再测真实解码影响。S17

真正的研究问题可以是：**在保持前缀约束和确定性 tie-break 的条件下，能否降低设备端验证预算分配的延迟与扩展成本？**这是一个具体候选题目，不是已确认的新颖性或已测得的收益。

17.3 把 controller 的目标从洁净吞吐扩展到真实有效产出

现有控制器连接 confidence 与运行时曲线，EPLB 使用历史负载。可以进一步研究：上下文长度、置信度校准误差、专家偏斜、统计/重排开销和服务时延约束，怎样共同影响验证预算。S16、S14

合理的比较至少包括固定 nextn、现有 VARLEN、以及新增策略；同时报告实际接受 token、错误/回退、完整墙钟吞吐与长尾。仅让理论 token/ms 更高，不足以证明系统更好。

阶段	先验证什么	建议产物
机制检查	状态、预算、顺序和数值是否等价	固定输入回归与边界用例
性能定位	哪一段真的在关键路径上?	trace、分段计时、显存曲线
单变量对照	某项改动单独带来什么?	同配置、多次重复结果
系统验收	有效 token、质量与长尾是否改善?	包含全部相关开销的对照报告

与前面的论文连起来看：TileSight 强调执行成本模型，Jev-Mem 强调把有限预算用在值得做的工作上，Megatron 报告强调布局与生命周期。DeepInfer 的价值，是让你在一个具体推理实现中看到这几类问题如何发生耦合。这里是方法论类比，不表示这些项目存在代码或算法继承关系。

18. 最后判断与源码阅读路线

最值得学的不是把每个优化开关都打开，而是弄清楚每个开关改变了哪一条依赖、哪一类存储和哪一种结果保证。

建议按下面顺序阅读，避免一开始就迷失在 kernel 文件中：

顺序	文件或模块	抓住的问题
1	README、examples README	交付边界和测量口径是什么?
2	config.py、environ.py	并行规模和最终配置怎样确定?
3	llm_engine.py、zos.cpp	请求状态如何跨迭代衔接?
4	Attention backend、attn.py	稀疏索引、布局和融合接口如何约束计算?
5	yoco.py 与 equivalence 测试	减少 query 后怎样维护缓存语义?
6	mlp.py、moe.py	当前拓扑到底走哪条 MoE 路径?
7	eplb/policy.py	热点、副本与 rank 放置如何选择?
8	dflash.py、mtp_varlen.hpp	草稿收益与执行成本怎样结合?
9	GPU varlen kernel、BI 测试	调度 kernel 与数值稳定性怎样验证?
10	benchmark、Pareto、eval	报告的数字到底回答哪个问题?

我的总体评价是：**这是研究“模型—运行时—算子联合设计”的高价值工程样本，但不是现成的通用服务平台，也不是一份已经完成严格跨框架、跨拓扑、真实接受率比较的性能证明。**

对你最直接的收获有三个：把确定性当作全栈契约；把草稿验证当作有成本的预算分配；把动态输入与静态图执行之间的元数据、缓存和状态转换当作一等优化对象。

附录：来源与阅读范围

所有 DeepInfer 源码链接均固定到同一提交，避免未来 main 分支变化后无法对应。文中“作者报告”指项目文档，公式整理和研究建议属于本文分析。未对 DeepGEMM/FlashMLA/DeepEP 的底层实现做完整递归审计。

S00 固定提交与版本边界
提交信息；不使用后续 main 分支状态替代该快照。

S01 README.md
发行范围、模型/平台、功能声明、图表实验规模与安装要求。

S02 deepinfer/init.py
包版本、平台延迟初始化与扩展装载。

S03 deepinfer/engine/llm_engine.py
重点阅读开头的引擎定义与 DeferredRelease，非整文件审计。

S04 deepinfer/config.py
ParallelConfig、CacheConfig 与 SchedulerConfig 相关范围。

S05 prepare_cp_mirror.py
wrapper 与前部 Python zigzag/reference 实现。

S06 v4_modules/attn.py
前部 Attention 类定义、融合条件及布局处理。

S07 layers/attention_backend.py
Sparse prefill/decode 的后端接口、head padding 与 BI 参数。

S08 v4_modules/yoco.py
producer/consumer 判断、边界 compact 与派生缓存失效。

S09 examples/yoco/README.md
L1/L2 验证流程、checkpoint 约束和 confidence 比较边界。

S10 deepinfer/environ.py
惰性环境变量、默认值、回退与 YoCo 约束。

S11 v4_modules/moe.py
逻辑/物理专家、shared expert 和 gate 参数容器。

S12 examples/README.md
离线使用、PPL、稳态测量、接受率计数、拓扑和 VE 计时范围。

S13 v4_modules/mlp.py
重点阅读前 580 行的权重、gate、Mega 与分离执行路径。

S14 eplb/policy.py
冗余副本分配与历史峰值目标下的贪心放置。

S15 models/mtp/dflash.py
类构造、confidence、main cache、采样与 decode graph 相关区间。

S16 csrc/engine/mtp_varlen.hpp
双历史队列、运行时估计与总验证预算搜索。

S17 cuda/schedule_mtp_varlen.py
累计概率、全比较排名与稳定 tie-break。

S18 csrc/engine/zos.cpp
前部输入准备、跨轮 remap 与容量检查；非完整文件审计。

S19 deepinfer/engine/arg_utils.py
按 engine type 设置 ZOS 默认值；另核对 worker 对应符号路径。

S20 deepinfer/worker/runtime_config.py
确定性算法开关、NPU BI 限制和 kernel 环境。

S21 tests/models/test_batch_invariant.py
使用 golden token 固定轨迹的批次不变性测试。

S22 model_executor/layers/linear.py
32-byte 行对齐、量化参数、scale 和 DeepGEMM 调用。

S23 .gitmodules
本次快照声明的外部算子与通信子模块。

S24 examples/sweep/plot_pareto.py
外部接受率、每用户/每卡坐标、重复点处理与一致性检查。

S25 eval/README.md
Pass@1 语义、生成/评分分离、完整覆盖与作者参考分数。

S26 NVIDIA：CUDA GPU Compute Capability
B200 与 H200 的计算能力分类；并不替代框架支持列表。

S27 build.sh
依赖安装、NCCL 替换与编译步骤。

S28 requirements-nvidia.txt
固定 PyTorch CUDA wheel 与 FlashAttention 依赖。

S29 examples/sweep/run_sweep.sh
扫描参数、PROFILE=0、prefill YoCo 默认及失败处理。